Lab 4 - Distributed-memory programming with MPI

Oguz Kaya
oguz.kaya@universite-paris-saclay.fr

Environment. This lab needs an MPI library and its launcher on a machine with several cores. Compiler Explorer compiles and runs a single program and cannot launch several processes, so the lab is done on your own machine, under Linux, Windows (through WSL) or macOS, with **Open MPI** installed: see the installation guide, `install-openmpi.pdf`. A program is compiled with `mpic++`, a wrapper around `g++`, and run with `mpirun`, which starts the processes:

`mpic++ -O2 program.cpp -o program` then `mpirun -np 4 ./program`

`-np 4` starts four processes; add `-oversubscribe` after `mpirun` to start more processes than your machine has cores (it works, no faster). Save your code regularly.

Submission. For the questions that ask for a comment or a result, write your answers as comments at the top of the corresponding source file, in the block provided for this purpose; the source files are the only thing you submit.

MPI programming. In this lab, we write our first programs with several processes, using the six functions of the lecture: `mpirun -np p` launches p processes that all execute the same program; a process knows its rank and the number of processes; two processes exchange data only by messages, a send on one side and a receive on the other.

`MPI_Init(&argc, &argv);`	starts MPI; the first MPI call of the program
`MPI_Finalize();`	stops MPI; the last MPI call of the program
`MPI_Comm_rank(MPI_COMM_WORLD, &rank);`	my rank k, from 0 to $p-1$
`MPI_Comm_size(MPI_COMM_WORLD, &size);`	the number p of processes
`MPI_Send(buf, count, type, dest, tag, MPI_COMM_WORLD);`	sends `count` elements of type `type` starting at `buf` to process `dest`
`MPI_Recv(buf, count, type, source, tag, MPI_COMM_WORLD, MPI_STATUS_IGNORE);`	receives into `buf` (room for `count` elements) the message of process `source`

These are the only MPI functions you need and are allowed to use in this lab. Remember the rules. Every variable belongs to the process that declares it: p processes, p copies, and no process can read or write another one's (there is no shared variable); the only way to use another process's data is a message. A message is an array of `count` elements of a basic type (`MPI_INT`, `MPI_FLOAT`, ...); a single variable is sent as an array of one element, by its address (`&x`, `count = 1`). A receive takes the first message that arrived from the process `source` with the label `tag` (0 everywhere in this lab); the messages from one process to another arrive in the order they were sent. `MPI_Send` returns at once (for the message sizes of this lab), `MPI_Recv` waits until the message has arrived.

Think of the data as a *global array distributed by blocks*: process k holds the block k, i.e. the elements $k\,n, \ldots, (k+1)\,n-1$ of a global array of $N = n\,p$ elements, in its local array `x[0..n-1]`; nobody holds the global array.

Part 1

Hello world with processes

A skeleton code is provided in the file `hello.cpp`; it contains the calls to `MPI_Init` and `MPI_Finalize`.

Ex. 1

a) Write the code in which each process prints one line

`I am process <k> of <p>`

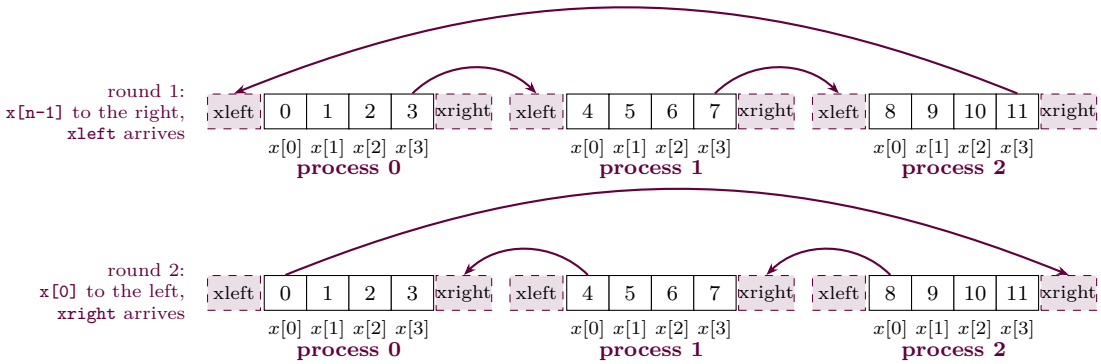
`<k>` being its rank and `<p>` the number of processes.

b) Run the program with `mpirun -np 1`, `-np 2`, `-np 4` and `-np 8`, twice each. What is printed? What changes from one run to the next, and what does not?

Part 2

1-D convolution on a distributed vector

We consider a vector $gx[0 \dots N-1]$ of $N = n\,p$ numbers distributed by blocks over the p processes: process k holds the elements $gx[kn], \dots, gx[(k+1)n-1]$ in its local array `x[0..n-1]`, and nobody holds the whole vector. We want to compute the *three-point average* of the vector, $y[i] = (gx[i-1] + gx[i] + gx[i+1])/3$ for every i, the simplest 1-D convolution (session 6); the vector is circular: the left neighbour of $gx[0]$ is $gx[N-1]$ and the right neighbour of $gx[N-1]$ is $gx[0]$. Each process computes its own block of y, in a local array `y[0..n-1]`. For the $n-2$ interior elements of its block, everything it needs is in its own `x`; for the first and the last, it needs one element of each neighbouring block, which another process holds: these two *frontier elements* must travel by messages, as the figure shows for $n = 4$ and three processes.



the cell i holds $gx[i]$, $N = 12$; the vector is circular: the left neighbour of $gx[0]$ is $gx[11]$

A skeleton code is provided in the file `convolution.cpp`, with $n = 4$. The global vector is given by a formula, `gx(i) = i * i`, so that every process can fill its own block itself, and the function `verify` checks each block of `y` against the formula.

Ex. 2

a) On paper, in the ANSWERS block: which process holds $gx[i]$, and in which cell of its `x`? Which element of gx is the `x[j]` of process k? Which processes are the left and right neighbours of process k, including for processes 0 and $p-1$?

b) Compute the ranks `left` and `right` of the two neighbours of the process.

c) Exchange the frontier elements in two rounds. Round 1: every process sends its last element `x[n-1]` to its right neighbour, then receives the last element of its left neighbour into `xleft`. Round 2: every process sends its first element `x[0]` to its left neighbour, then receives the first element of its right neighbour into `xright`.

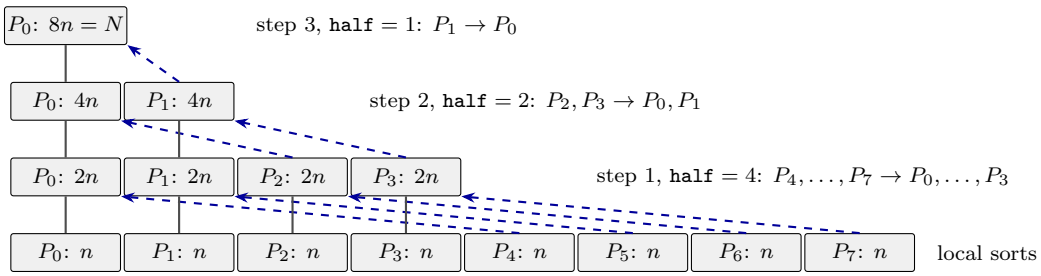
d) Compute `y[j]` for $j = 0, \dots, n-1$, with `xleft` in place of `x[-1]` and `xright` in place of `x[n]`; the skeleton prints the blocks and checks `y`.

e) Run the program with `-np 1`, `2`, `4` and `8`: does `verify` say OK every time? Which elements of `y` depend on another process's block, and how many messages does each process send in all? Why is the exchange done in two rounds (a send, a receive, a send, a receive) rather than the two sends first and then the two receives: what could go wrong with `-np 2`?

Part 3

Parallel mergesort with processes

Mergesort (lab 2) sorts an array by sorting its two halves, recursively, then *merging* the two sorted halves into one sorted array; merging two sorted arrays is easy, one repeatedly takes the smaller of the two front elements. We sort an array of $N = 2^{22}$ integers with p processes, p a power of two $(1, 2, 4, 8, \dots)$ and N a multiple of p, distributed by blocks: process k holds the block k, $n = N/p$ pseudo-random integers, in `a[0..n-1]`. Every process first sorts its own block, which needs no communication; the sorted blocks are then merged in $\log_2 p$ steps, until process 0 holds the whole sorted array: at each step, the *second half* of the active processes folds onto the *first half*, and only the first half stays active. The figure shows the steps for $p = 8$: a dashed arrow is a message carrying a sorted block; a plain line means that the process keeps its elements and merges the block it receives with them.



The rule of the merging steps: at each step, the active processes are $0, \ldots, 2\,\texttt{half} - 1$, with $\texttt{half} = p/2$ at the first step, and every active process holds m sorted elements in `a[0..m-1]`, $m = n$ at the first step. A process of rank $\geq$ `half` sends its m elements to process `rank` $-$ `half`, and its work is done. A process of rank $<$ `half` receives the m elements of process `rank` $+$ `half` and places them right after its own, in `a[m..2m-1]`, then merges the two sorted halves of `a[0..2m-1]`: it now holds $2m$ sorted elements. Then `half` is halved and the next step begins, with half as many active processes; after the step with `half` $= 1$, process 0 holds the N elements. A skeleton code is provided in the file `mergesort.cpp`. It allocates on every process the arrays `a` and `temp` with room for the N elements of the whole array, fills the block of the process, and provides the functions of lab 2:

- `mergesort(a, temp, n)` sorts the n elements of the array `a` sequentially, using `temp`, a scratch buffer of n elements;
- `merge(a, temp, n)` takes an array `a` of n elements whose two halves `a[0..n/2)` and `a[n/2..n)` are *already sorted*, and rearranges `a` into a single sorted array of n elements, using `temp` as scratch buffer;
- `verify(a, N, p, t_par)`, to be called by process 0 at the end: it regenerates the whole input, sorts it sequentially (and times it), and compares it element by element with `a`: equal means that `a` is sorted *and* contains exactly the initial elements; it prints the verdict, the sequential time and the speedup.

Recall that `a + m` is the address of `a[m]`, so that the part of `a` starting at index m can be passed to a function, or given to `MPI_Recv`, as an array of its own.

Ex. 3

a) Sort the block of the process, `a[0..n-1]`, with `mergesort`.

b) Write the merging steps: a loop on $\texttt{half} = p/2, p/4, \ldots, 1$, in which the process either sends its block to process `rank` $-$ `half` and leaves the loop, or receives the block of process `rank` $+$ `half` and merges, following the rule above. At the end, process 0 holds the sorted array in `a[0..N-1]`.

c) Process 0 checks the result with `verify`.

d) Run the program with `-np 1`, `2`, `4` and `8` (on your own machine, with at least 4 cores for the last two): note the sequential and parallel times and the speedups. Why is the speedup below the number of processes: which part of the work is not done in parallel, and how much data travels through the messages?